

NEXUS

Nonlinear Exploratory Framework for X-domain Unified Systems

Technical Reference & Developer Guide

Beta Version 0.1 — May 2026

Samyak Jain

National Institute of Technology Rourkela

github.com/SAMYAK-2006

*“Any bounded dynamical system has a statistical attractor.
Regime stress is departure from that attractor.
Regime shift is escape to a new attractor basin.
The data-trust score $\alpha(t)$ measures how far and how fast
the system is departing from what it knows.”*

— SMA Framework (Jain, 2026)

Built on: Statistical Manifold Attractor Framework · Koopman Operator Theory · SDE Regime
Switching · Topological Data Analysis

Contents

1	Overview	3
1.1	Design Principles	3
2	Architecture	3
2.1	System Layers	3
2.2	File Structure	4
3	Mathematical Framework	4
3.1	The Statistical Feature Map Φ	4
3.2	The Statistical Parameter Manifold	5
3.3	Trajectory Kinematics	5
3.4	The Statistical Attractor	6
3.5	The Data-Trust Score $\alpha(t)$	6
3.6	Regime Detection and Escape Phases	6
3.7	Stress Dimensionality	7
4	Applicability Diagnostics	7
5	Statistical Analysis Engine	7
5.1	Stationarity and Memory	8
5.2	Nonlinearity Tests	8
5.3	Spectral Analysis	8
5.4	Risk Metrics	8
5.5	Changepoint Detection	8
6	Prediction Engine	8
6.1	Model Inventory	8
6.1.1	ARIMA	9
6.1.2	OU-SDE	9
6.1.3	Koopman/DMD	9
6.1.4	SMA Gradient Boosting	9
6.2	Trust-Weighted Ensemble	9
6.3	Forecast Intervals	10
7	Operating Modes	10
7.1	Mode 1 — Batch Analysis	10
7.1.1	Output Report Sections	10
7.2	Mode 2 — Real-Time Streaming	11
7.2.1	Real-Time Performance	11
8	Installation and Quick Start	11
8.1	Requirements	11
8.2	Quick Start	12
8.3	Using Your Own Data	12
9	Beta Status and Roadmap	12
9.1	What Works in v0.1	12

9.2	Known Limitations	13
9.3	Planned for v0.2	13
10	Theoretical Foundations	13
10.1	Statistical Manifold Attractor Framework	13
10.2	Koopman Operator Theory	13
10.3	Ornstein-Uhlenbeck Process	13
10.4	Dynamic Mode Decomposition with Control	14
11	API Reference	14
11.1	GeometricEngine	14
11.2	TimeSeriesAnalyzer	14
11.3	EnsemblePredictor	14
11.4	RealTimeEngine	15

1. Overview

NEXUS is a domain-agnostic engine for the analysis, characterization, and prediction of nonlinear stochastic time series. It takes as input any univariate time series — financial, physiological, physical, or synthetic — and produces as output a comprehensive analysis report together with a trust-weighted ensemble forecast.

The system is organized around a central question that classical time series methods do not answer directly:

Central Question

Where in statistical parameter space is this system? How fast is it moving? Toward what does it return? And how much should we trust its historical patterns right now?

This reframes analysis from *label-and-classify* to *geometry-and-kinematics*, following the Statistical Manifold Attractor (SMA) framework introduced in Jain (2026).

1.1 Design Principles

1. **Universality before domain.** The same pipeline runs on S&P 500 returns, EEG recordings, turbofan sensor data, and epidemic curves. Domain adaptation is a parameter, not a redesign.
2. **Applicability-first.** Before computing anything, NEXUS checks whether the SMA attractor framework is theoretically warranted for the input series. It degrades gracefully to geometric-only mode when the attractor does not exist.
3. **Trust-weighted prediction.** Every forecast is modulated by $\alpha(t)$, a continuous data-trust score derived from manifold geometry. When the system is in an escape phase, structural priors dominate.
4. **Modular prediction engine.** Each forecasting model is a self-contained module. New models drop in without touching the rest of the system.
5. **Two operating modes.** Batch mode (CSV/Excel input $\rightarrow$ HTML report) and real-time mode (streaming input $\rightarrow$ rolling forecasts).

2. Architecture

2.1 System Layers

NEXUS is organized into four sequential layers plus two operating modes:

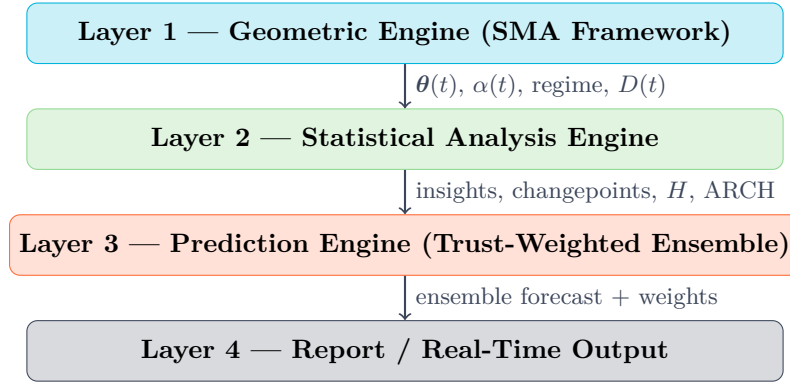


Figure 1: NEXUS four-layer pipeline.

2.2 File Structure

```

1  nexus/
2      run_nexus.py                # Entry point      run this
3      requirements.txt
4      nexus/
5          __init__.py
6          core/
7              __init__.py
8              geometric.py        # Layer 1: SMA engine
9          analysis/
10             __init__.py
11             analyzer.py         # Layer 2: Statistical
12 analysis
13     prediction/
14         __init__.py
15         engine.py               # Layer 3: Prediction engine
16 report/
17     __init__.py
18     generator.py               # Layer 4: HTML report
19 realtime/
20     __init__.py
21     engine.py                  # Real-time streaming engine

```

Listing 1: Repository layout

3. Mathematical Framework

3.1 The Statistical Feature Map Φ

Let $X = (X_1, X_2, \dots, X_T)$ be a scalar time series. For a rolling window of width W ending at time t , define the **observation window**:

$$X_{t:W} = (X_{t-W}, X_{t-W+1}, \dots, X_{t-1}) \in \mathbb{R}^W.$$

The **statistical feature map** $\Phi : \mathbb{R}^W \rightarrow \mathbb{R}^k$ extracts k statistical properties from each window:

Definition 3.1 (Statistical Feature Vector). The statistical feature vector at time t is

$$\boldsymbol{\theta}(t) = \Phi(X_{t:W}) \in \mathbb{R}^k,$$

where $k = 14$ in the current implementation. The features are:

Table 1: Feature map Φ — components of $\boldsymbol{\theta}(t)$

Index	Symbol	Name	Formula	Interpretation
0	μ_t	Mean	$\frac{1}{W} \sum_i X_i$	Level
1	σ_t^2	Variance	$\frac{1}{W-1} \sum_i (X_i - \mu_t)^2$	Volatility
2	γ_t	Skewness	$\frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3}$	Asymmetry
3	κ_t	Exc. Kurtosis	$\frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4} - 3$	Tail weight
4	$\rho_t^{(1)}$	ACF lag-1	$\text{Corr}(X_i, X_{i+1})$	Short memory
5	$\rho_t^{(2)}$	ACF lag-2	$\text{Corr}(X_i, X_{i+2})$	Short memory
6	$\rho_t^{(5)}$	ACF lag-5	$\text{Corr}(X_i, X_{i+5})$	Medium memory
7	ν_t	Vol of abs	$\text{Std}(X_i)$	Vol clustering
8	$\sigma_{\downarrow, t}^2$	Downside var	$\text{Var}(X_i \mid X_i < 0)$	Tail risk
9	MAD_t	Mean abs dev	$\frac{1}{W} \sum_i X_i - \mu_t $	Robust spread
10	DD_t	Max drawdown	$\max_i \frac{\prod_{\max_{j \leq i}} - \prod_i}{\prod_{\max_{j \leq i}}}$	Tail event
11	G_t	Gain/pain	$\frac{\mathbb{E}[X X>0]}{\mathbb{E}[X X<0]}$	Risk-reward
12	H_t	Hurst exp	R/S analysis	Long memory
13	IQR_t	Interquartile range	$Q_{0.75} - Q_{0.25}$	Robust spread

3.2 The Statistical Parameter Manifold

Definition 3.2 (Parameter Manifold). The **statistical parameter manifold** is the image of Φ :

$$\Theta = \{\boldsymbol{\theta} \in \mathbb{R}^k : \boldsymbol{\theta} = \Phi(\mathbf{x}) \text{ for some } \mathbf{x} \in \mathbb{R}^W\} \subseteq \mathbb{R}^k.$$

Under mild regularity, Θ is a smooth submanifold of $\mathbb{R}^k$.

In practice, $\boldsymbol{\theta}(t)$ is standardized to zero mean and unit variance before all downstream computations. PCA reduction to $m \leq k$ dimensions gives $\hat{\boldsymbol{\theta}}(t) \in \mathbb{R}^m$ for visualization and regime detection.

3.3 Trajectory Kinematics

Definition 3.3 (Velocity, Speed, Acceleration).

$$\mathbf{v}(t) = \boldsymbol{\theta}(t) - \boldsymbol{\theta}(t-1) \quad (\text{velocity}) \quad (1)$$

$$s(t) = \|\mathbf{v}(t)\|_2 \quad (\text{speed}) \quad (2)$$

$$\mathbf{a}(t) = \mathbf{v}(t) - \mathbf{v}(t-1) \quad (\text{acceleration}) \quad (3)$$

Definition 3.4 (Net Displacement and Persistence). For displacement window W_d :

$$\mathbf{d}(t, W_d) = \boldsymbol{\theta}(t) - \boldsymbol{\theta}(t - W_d), \quad \hat{\mathbf{d}}(t, W_d) = \frac{\mathbf{d}(t, W_d)}{\|\mathbf{d}(t, W_d)\|_2}.$$

The **displacement persistence score** is the cosine similarity between past and future displacement:

$$P(t, W_d) = \langle \hat{\mathbf{d}}(t, W_d), \hat{\mathbf{d}}(t + W_d, W_d) \rangle \in [-1, 1].$$

Interpretation of $P(t, W_d)$

- $P > 0$: **momentum** — trajectory continues in the same direction
- $P = 0$: **random walk** — no directional memory
- $P < 0$: **mean reversion** — trajectory reverses direction

The **Universal Attractor Theorem** (Jain, 2026) proves that for any bounded system, $\mathbb{E}[P(t, W_d)] \leq 0$ for large W_d .

3.4 The Statistical Attractor

Definition 3.5 (Attractor and Distance). The **statistical attractor** is the ergodic mean:

$$\boldsymbol{\theta}^* = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \boldsymbol{\theta}(t) = \mathbb{E}_\pi[\boldsymbol{\theta}(t)].$$

The **attractor distance** and **normalized stress index** are:

$$D(t) = \|\boldsymbol{\theta}(t) - \boldsymbol{\theta}^*\|_2, \quad \hat{D}(t) = \frac{D(t)}{\hat{q}_{0.95}(t)},$$

where $\hat{q}_{0.95}(t)$ is the running 95th percentile of D .

In the real-time engine, $\boldsymbol{\theta}^*$ is estimated via an exponential moving average:

$$\boldsymbol{\theta}_t^* = \lambda \boldsymbol{\theta}_{t-1}^* + (1 - \lambda) \boldsymbol{\theta}(t), \quad \lambda = 0.99,$$

which allows the attractor to migrate slowly under structural change.

3.5 The Data-Trust Score $\alpha(t)$

Definition 3.6 (Data-Trust Score).

$$\alpha(t) = \sigma\left(-\beta_1 \hat{D}(t) - \beta_2 \max(\dot{D}(t), 0)\right) \in (0, 1),$$

where $\sigma(x) = (1 + e^{-x})^{-1}$ is the logistic function, $\dot{D}(t) = D(t) - D(t-1)$ is the stress rate, and $\beta_1, \beta_2 > 0$ are calibration parameters (defaults: $\beta_1 = 2.0$, $\beta_2 = 1.0$).

Table 2: Trust score $\alpha(t)$ interpretation

$\alpha(t)$ range	State	Effect on ensemble
> 0.70	Near attractor, stable	Data-driven models weighted up
$0.40\text{--}0.70$	Intermediate stress	Balanced weighting
< 0.40	Escape phase / crisis	Structural priors (OU) dominant

3.6 Regime Detection and Escape Phases

Regimes are identified by quantile thresholding on $D(t)$:

Table 3: Regime labels

Label	Condition	Interpretation
0	$D(t) \leq Q_{33}$	Calm / near-attractor
1	$Q_{33} < D(t) \leq Q_{66}$	Intermediate
2	$D(t) > Q_{66}$	Stressed / far from attractor

An **escape phase** (Proposition 3.2, Jain 2026) occurs when all three conditions hold simultaneously:

1. $D(t) > Q_{90}$ (far from attractor)
2. $\dot{D}(t) > 0$ (distance increasing)
3. $P(t, W_d) > 0$ (momentum locally dominant)

3.7 Stress Dimensionality

During escape phases, the trajectory concentrates on a low-dimensional subspace. The **stress dimensionality** (participation ratio) measures this:

$$\text{sdim}(t) = \frac{(\sum_i \lambda_i(t))^2}{\sum_i \lambda_i(t)^2} \in [1, m],$$

where $\lambda_1(t) \geq \dots \geq \lambda_m(t)$ are the eigenvalues of the local covariance of $\hat{\theta}$ over a window $[t - L, t + L]$. $\text{sdim}(t) \rightarrow 1$ signals dimensionality collapse — the empirical signature of crisis.

4. Applicability Diagnostics

NEXUS always checks whether the SMA attractor framework is applicable before proceeding. This is critical: the attractor is not universal and must be diagnosed empirically.

Algorithm 1 SMA Applicability Check

Require: time series X , window W , minimum length $n_{\min}$

- 1: **Length check:** sufficient $\leftarrow (|X| \geq n_{\min})$
 - 2: **Boundedness:** compute range ratio $\rho = \text{range}(X_{T/2:T}) / \text{range}(X_{1:T/2})$; bounded $\leftarrow (\rho < 3)$
 - 3: **Stationarity:** run ADF test; record p_{ADF}
 - 4: **Compute $\theta(t)$ matrix via Φ**
 - 5: **Persistence test:** compute $P(t, W_d)$ for all valid t ; run t -test $H_0 : \mathbb{E}[P] = 0$ vs $H_1 : \mathbb{E}[P] < 0$
 - 6: $\text{mean_reverting} \leftarrow (\hat{\mathbb{E}}[P] < 0) \wedge (p < 0.1)$
 - 7: $\text{attractor_exists} \leftarrow \text{mean_reverting} \wedge \text{bounded}$
 - 8: **Return** ApplicabilityReport with all flags and notes
-

Graceful Degradation

If `attractor_exists = False`, NEXUS enters **geometric-only mode**: all kinematic quantities ($s(t)$, $D(t)$, $\alpha(t)$, regimes) are still computed and reported, but the trust score and attractor distance are labelled as descriptive rather than theory-guaranteed. The prediction engine still runs all models; only the trust-weighting logic is affected.

5. Statistical Analysis Engine

Layer 2 runs a comprehensive battery of tests and analyses on the raw series. Each category and its implementation:

5.1 Stationarity and Memory

- **ADF test** (Augmented Dickey-Fuller): H_0 : unit root. $p < 0.05 \Rightarrow$ stationary.
- **Hurst exponent** H via R/S analysis over logarithmically spaced lags. $H > 0.5$: persistent/trending. $H < 0.5$: mean-reverting. $H = 0.5$: random walk.
- **Sample entropy**: measures regularity of the series. Low $\Rightarrow$ regular/predictable.
- **Lyapunov exponent** (approximate): positive value indicates chaotic sensitivity to initial conditions, estimated via divergence of nearest-neighbour trajectories in delay-embedding space.

5.2 Nonlinearity Tests

- **Time reversibility**: $\mathbb{E}[(X_{t+h} - X_t)^3] \neq 0$ implies the series is statistically irreversible — a necessary signature of nonlinear dynamics.
- **BDS-proxy**: correlation-integral proxy for independence of residuals. Significant value $\Rightarrow$ nonlinear dependence.
- **ARCH test**: tests for volatility clustering (H_0 : no ARCH effects) via Engle’s LM test or Pearson r on squared values.
- **Teräsvirta proxy**: neural-network nonlinearity test via F -statistic comparing linear vs cubic regression.

5.3 Spectral Analysis

- **Periodogram**: dominant frequency f^* and associated period $T^* = 1/f^*$.
- **Spectral entropy**: $H_s = -\sum_f p(f) \log p(f)$ normalized to $[0, 1]$. Near 0 $\Rightarrow$ periodic. Near 1 $\Rightarrow$ white noise.
- **Complexity classification**: periodic (top-3 frequencies explain $> 70\%$ of power), noisy ($H_s > 0.85$), chaotic ($H_s \in [0.5, 0.85]$), mixed otherwise.
- **Spectrogram**: short-time Fourier transform for time-varying spectral structure.

5.4 Risk Metrics

- Realized volatility, volatility-of-volatility (vol clustering proxy)
- Maximum drawdown, Calmar ratio, approximate Sharpe ratio
- Value at Risk (VaR_{95}) and Conditional VaR (CVaR_{95})
- Tail ratio (95th / 5th percentile of absolute returns)

5.5 Changepoint Detection

NEXUS uses `ruptures.Pelt` with RBF kernel if available, falling back to a CUSUM-based detector. The penalty parameter is set to 3 (generous — prefers recall over precision for anomaly awareness).

6. Prediction Engine

6.1 Model Inventory

Four base forecasters are implemented, each as a self-contained class inheriting from `BaseForecaster`:

Table 4: Prediction engine — base models

Class	Model	Key idea	Best regime
ARIMAForecaster	Auto-ARIMA	Box-Jenkins, auto-selects (p, d, q)	Stationary, linear
OUForecaster	OU-SDE	AR(1) $\rightarrow$ OU parameters, analytical CI	Mean-reverting, stressed
KoopmanForecaster	DMD/Koopman	Hankel embedding + SVD-based DMD	Nonlinear, structured
SMAGBForecaster	SMA-GradBoost	Gradient boosting on $\{D(t), \alpha(t), s(t), P(t), \hat{\theta}\}$	High $\alpha(t)$, feature-rich

6.1.1 ARIMA

Auto-selects integration order d via ADF test, then grid-searches $(p, q) \in \{0, \dots, 3\}^2$ by AIC. Falls back to mean forecast on failure.

6.1.2 OU-SDE

Fits an AR(1) regression $X_t = a + bX_{t-1} + \varepsilon_t$ and maps to OU parameters:

$$\kappa = -\frac{\log b}{\Delta t}, \quad \theta^* = \frac{a}{1-b}, \quad \sigma_{\text{OU}} = \hat{\sigma}_\varepsilon \sqrt{2\kappa}.$$

Analytical conditional mean and variance:

$$\mathbb{E}[X(t+h) \mid X(t)] = \theta^* + (X(t) - \theta^*)e^{-\kappa h},$$

$$\text{Var}[X(t+h) \mid X(t)] = \frac{\sigma_{\text{OU}}^2}{2\kappa} (1 - e^{-2\kappa h}).$$

6.1.3 Koopman/DMD

Constructs a Hankel delay-embedding matrix of dimension d (Taken’s theorem), applies SVD-truncated Dynamic Mode Decomposition, and evolves the DMD modes forward:

$$\hat{X}(t+h) = \text{Re} \left[\Phi \cdot (b \odot \lambda^h) \right]_1,$$

where $\Phi \in \mathbb{C}^{d \times r}$ are DMD modes, $\lambda \in \mathbb{C}^r$ are eigenvalues, and b are initial amplitudes fit by least squares.

6.1.4 SMA Gradient Boosting

Constructs a feature matrix from the SMA outputs at each timestep:

$$f(t) = \left[D(t), \dot{D}(t), \alpha(t), s(t), P(t, W_d), \text{sdim}(t), \hat{\theta}_{1:3}(t) \right] \in \mathbb{R}^9.$$

A **MultiOutputRegressor** wrapping **GradientBoostingRegressor** is trained to predict all h future values jointly. This is the “geometry-informed” model: it explicitly uses manifold features as predictors.

6.2 Trust-Weighted Ensemble

The ensemble weight for model m is:

$$w_m^{\text{base}} = \frac{1/\text{RMSE}_m}{\sum_{m'} 1/\text{RMSE}_{m'}},$$

then modulated by the current trust score $\alpha(t)$:

$$w_m = \begin{cases} w_m^{\text{base}} \cdot (1 + 2(1 - \alpha(t))) & \text{if } m \in \text{structural models (OU)} \\ w_m^{\text{base}} \cdot (0.5 + 0.5\alpha(t)) & \text{otherwise} \end{cases}$$

after renormalization to sum to 1. The ensemble point forecast and intervals are the corresponding weighted sums.

Trust-Weighting Intuition

When $\alpha(t) \approx 1$ (system near attractor, stable history), data-driven models (ARIMA, DMD, GBM) receive full weight. When $\alpha(t) \approx 0$ (escape phase, historical patterns unreliable), the OU structural prior is boosted to $3\times$ its base weight.

6.3 Forecast Intervals

All models produce 80% and 95% prediction intervals. For parametric models, intervals grow as $\sqrt{h}$ with horizon (uncertainty accumulation):

$$\hat{X}(t+h) \pm z_\alpha \cdot \hat{\sigma} \cdot \sqrt{h},$$

where $\hat{\sigma}$ is the in-sample residual standard deviation and $z_{0.80} = 1.282$, $z_{0.95} = 1.960$.

7. Operating Modes

7.1 Mode 1 — Batch Analysis

Input: a CSV or Excel file. Output: a self-contained HTML report.

```

1  # Demo (synthetic data, no file needed)
2  python run_nexus.py --demo
3
4  # Basic      auto-detects numeric column
5  python run_nexus.py --file data.csv
6
7  # Full control
8  python run_nexus.py \
9      --file      data.csv \
10     --column     close \
11     --window     60 \
12     --horizon    30 \
13     --output     my_report.html
```

Listing 2: Batch mode usage

7.1.1 Output Report Sections

1. **SMA Applicability:** all diagnostic flags, notes, persistence test result
2. **Series Overview:** raw series with regime coloring, escape phase markers, speed, trust, attractor distance, persistence
3. **Geometric Portrait:** manifold trajectory (PC1 vs PC2), stress dimensionality, regime occupancy, PCA variance, acceleration

4. **Statistical Analysis:** distribution, ACF, squared-ACF, rolling stats, Q-Q plot, Hurst R/S
5. **Key Insights:** auto-generated interpretive statements from all test results
6. **Symmetry & Spectral:** PSD, spectrogram, phase portrait, return map, time reversibility, rolling Z-score
7. **Forecast:** historical + ensemble forecast + 80/95% CI + individual model forecasts + model weight table

7.2 Mode 2 — Real-Time Streaming

Input: historical initialization series + live stream of new values. Output: a `RealtimeSnapshot` per tick.

```

1  from nexus.realtime.engine import RealTimeEngine
2  import numpy as np
3
4  # Initialize with history
5  engine = RealTimeEngine(
6      window=60,
7      horizon=20,
8      buffer_size=1000,
9      refit_interval=50,    # refit models every 50 ticks
10 )
11 engine.initialize(historical_series)
12
13 # Streaming loop      call update() on each new tick
14 for new_value in live_stream:
15     snap = engine.update(new_value)
16
17     print(f"t={snap.n_observations}")
18     print(f"  Regime      : {snap.regime}")
19     print(f"  Trust a(t)  : {snap.trust_score:.4f}")
20     print(f"  Speed s(t)   : {snap.speed:.4f}")
21     print(f"  Escape?     : {snap.is_escape_phase}")
22     print(f"  Forecast    : {snap.forecast[:5]}")

```

Listing 3: Real-time engine usage

7.2.1 Real-Time Performance

The real-time engine is designed for sub-millisecond per-tick latency:

- Geometric quantities ($s(t)$, $D(t)$, $\alpha(t)$, $P(t)$) recomputed every tick using rolling buffers
- Attractor θ^* updated via EMA (no full recompute)
- Prediction models refitted every `refit_interval` ticks (default 50); cached forecast shifted between refits
- Regime thresholds updated every 100 ticks

8. Installation and Quick Start

8.1 Requirements

```
1 pip install numpy pandas scipy scikit-learn statsmodels matplotlib
   ruptures
```

Python ≥ 3.10 required. All dependencies are standard scientific Python; no deep learning framework is needed for the base system.

8.2 Quick Start

```
1 # 1. Clone
2 git clone https://github.com/SAMYAK-2006/nexus.git
3 cd nexus
4
5 # 2. Install
6 pip install -r requirements.txt
7
8 # 3. Test immediately
9 python run_nexus.py --demo
10
11 # 4. Open the report
12 # nexus_report.html is created in the current directory
13 # Open in any browser
```

8.3 Using Your Own Data

NEXUS accepts any CSV or Excel file with at least one numeric column:

```
1 # Auto-detect column
2 python run_nexus.py --file your_data.csv
3
4 # Specify column
5 python run_nexus.py --file prices.csv --column close --horizon 30
```

Window Size Guidance

The window W should be set relative to the natural timescale of the domain. As a rule: $W \approx 2\text{--}4 \times$ the expected regime duration. Default: $W = 60$. For very short series (< 200 observations), NEXUS auto-adjusts $W = \lfloor n/5 \rfloor$.

9. Beta Status and Roadmap

9.1 What Works in v0.1

- Full SMA geometric pipeline (feature map, kinematics, attractor, trust score, regimes, escape detection)
- Complete statistical analysis battery (14 tests/metrics)
- Four-model trust-weighted ensemble (ARIMA, OU-SDE, Koopman-DMD, SMA-GradBoost)
- Self-contained HTML report with 5 visualization dashboards
- Real-time streaming engine with per-tick regime and trust updates
- Graceful degradation when attractor does not exist

9.2 Known Limitations

- **Univariate only:** multivariate extension is mathematically straightforward (stack feature maps across channels) but not yet implemented
- **No GARCH:** volatility forecasting via GARCH/EGARCH is a natural next module
- **No Fisher-Rao metric:** all distances use Euclidean norm on standardized features; the natural metric on Θ is the Fisher-Rao metric (information geometry) — a principled refinement
- **Attractor migration:** the time-varying attractor extension (Definition 3.2, Jain 2026) is partially implemented via EMA in real-time mode but not yet in batch mode
- **No deep sequence models:** RWKV/Transformer forecaster is listed in the SMA paper as future work; hook exists in `BaseForecaster` for easy addition
- **Benchmark validation:** systematic backtesting against BOCPD, PELT, and HMM baselines on real datasets is pending

9.3 Planned for v0.2

1. Multivariate input support
2. GARCH volatility module
3. Fisher-Rao metric option
4. Hierarchical attractor structure (sector/fleet-level SMA)
5. RWKV/Transformer forecasting plugin
6. Systematic benchmark suite (BOCPD, PELT, HMM, LSTM)
7. Real dataset validation: S&P 500, EEG (PhysioNet), NASA CMAPSS

10. Theoretical Foundations

NEXUS is built on four bodies of theory:

10.1 Statistical Manifold Attractor Framework

Jain (2026) establishes the Universal Attractor Theorem: for any bounded ergodic process, $\mathbb{E}[P(t, W)] \leq 0$ for large W , with equality only for periodic processes. This is proved from first principles (compactness of Θ) without parametric assumptions. The framework is validated across finance, solar physics, epidemiology, and climatology.

10.2 Koopman Operator Theory

The Koopman operator $\mathcal{K}$ lifts nonlinear dynamics into a linear evolution on an infinite-dimensional function space. DMD (Schmid, 2010; Tu et al., 2014) provides a finite-rank approximation from data. PyKoopman (Pan et al., 2024) provides a standard implementation; NEXUS implements its own SVD-based DMD directly for dependency minimization.

10.3 Ornstein-Uhlenbeck Process

The OU process $d\theta = \kappa(\theta^* - \theta) dt + \sigma dW$ is the continuous-time analogue of AR(1) and the canonical mean-reverting SDE. NEXUS uses it as both a forecasting model and as the theoretical representation of the SMA mean-reversion result (Proposition 3.1, Jain 2026).

10.4 Dynamic Mode Decomposition with Control

Proctor et al. (2016) extend DMD to systems with inputs. The Hankel-DMD variant used here (delay coordinates via Taken’s embedding theorem) handles systems where the full state is not directly observed.

11. API Reference

11.1 GeometricEngine

```

1 from nexus.core.geometric import GeometricEngine
2
3 engine = GeometricEngine(
4     window=60,          # rolling window W
5     pca_components=3,    # PCA dimensions m
6     displacement_window=None, # W_d (default: W//3)
7     trust_beta1=2.0,     # beta_1 in alpha(t)
8     trust_beta2=1.0,     # beta_2 in alpha(t)
9     regime_n_clusters=3, # number of regimes
10    escape_percentile=0.90,
11 )
12
13 result = engine.fit_transform(series) # -> SMAResult

```

SMAResult fields: theta, theta_pca, velocity, speed, acceleration, acceleration_mag, displacement, persistence, attractor, attractor_dist, stress_norm, stress_rate, trust_score, regime_labels, escape_phases, stress_dim, pca_variance_ratio, applicability.

11.2 TimeSeriesAnalyzer

```

1 from nexus.analysis.analyzer import TimeSeriesAnalyzer
2
3 analyzer = TimeSeriesAnalyzer(freq='D', returns_mode=False)
4 result = analyzer.analyze(series) # -> AnalysisResult

```

AnalysisResult fields include: hurst, lyapunov_approx, sample_entropy, time_reversibility, arch_effect, spectral_complexity, n_changepoints, key_insights, and all standard statistics.

11.3 EnsemblePredictor

```

1 from nexus.prediction.engine import EnsemblePredictor
2
3 predictor = EnsemblePredictor()
4 forecast = predictor.predict(
5     series=series,
6     horizon=20,
7     sma_result=sma_result,
8     current_trust=0.75,
9     current_regime=0,
10 )
11 # forecast.point_forecast -> np.ndarray (horizon,)
12 # forecast.model_weights -> dict[str, float]

```

11.4 RealTimeEngine

```
1 from nexus.realtime.engine import RealTimeEngine
2
3 engine = RealTimeEngine(window=60, horizon=20)
4 engine.initialize(historical_series)
5 snap = engine.update(new_value)
6 # snap.trust_score, snap.regime, snap.forecast, ...
```

References

-
- [1] Jain, S. (2026). *Statistical Manifold Geometry of Regime Dynamics: A Universal Attractor Framework*. Preprint, NIT Rourkela.
 - [2] Schmid, P.J. (2010). Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656, 5–28.
 - [3] Pan, S., et al. (2024). PyKoopman: A Python package for data-driven approximation of the Koopman operator. *Journal of Open Source Software*, 9(94).
 - [4] Proctor, J.L., Brunton, S.L., & Kutz, J.N. (2016). Dynamic mode decomposition with control. *SIAM Journal on Applied Dynamical Systems*, 15(1), 142–161.
 - [5] Uhlenbeck, G.E., & Ornstein, L.S. (1930). On the theory of the Brownian motion. *Physical Review*, 36(5), 823–841.
 - [6] Adams, R.P., & MacKay, D.J.C. (2007). Bayesian online changepoint detection. *arXiv:0710.3742*.
 - [7] Killick, R., Fearnhead, P., & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. *JASA*, 107(500), 1590–1598.
 - [8] Brunton, S.L., & Kutz, J.N. (2019). *Data-Driven Science and Engineering*. Cambridge University Press.